REPORT

Explain your design choices and justify command selection.

In the various tasks proposed for this assessment, a very similar approach has been adopted, basing the main structure on Bash due to its native ability to interact with the Operating System for the required actions. Although similar results could be achieved with Python, the efficiency provided by Bash allows for the necessary actions in most tasks to be performed with great precision. On the other hand, Python's use has been focused on binary comparison tasks and file type validation, where Bash lacks the necessary precision.

To carry out the different actions required to successfully complete the proposed tasks, a workflow has been designed where each function within each script fulfills each of the actions specified in the tasks; the code is accompanied by a series of basic comments to help guide any third party reviewing it. A generic structure of functions composed of conditional loops is used, seeking to unify different command results, with a structure when writing loops that aims to mitigate errors and invalid data entries for the program. The idea is to propose an interactive menu, listing the different possible actions, including a mandatory script exit action, allowing a constant return to the starting point (main menu), thus strengthening the code against errors. To achieve these codes, certain commands and parameters that may be more complex to understand have also been used, such as the following examples:

- `sort -t"|" -k4, 4nr`**:** This command can be found in the Task 1 script. It allows reordering the list of pending jobs according to their priority, ensuring that critical tasks are not lost.
- `((remaining = est - TIME_QUANTUM))`**:** Within the Round Robin algorithm, this Bash arithmetic operation allows calculating how much execution time is left for a job after consuming its TIME_QUANTUM, which is approximately 5 seconds.
- `trap cleanup SIGINT SIGTERM`**:** This command is vital for resilience. It captures interruptions (such as Ctrl+C) to execute a cleanup function that removes

.lock files. Without this, the system could remain permanently blocked after an unexpected shutdown.

- `awk -F"|" '{printf "%-10s %-15s", $1, $2}'`**:** Used for a more professional visualization of the queue. It transforms raw pipe-separated data into an aligned and readable table.

Similarly, if any doubt remains regarding the functionality of any command or parameter, the Linux manuals available on all operating systems can be used. They can be accessed with the following command: `man <command>`.

Advanced Operating System Concepts

Process Management & Scheduling

To complete the different tasks, two of the most widely used scheduling algorithms have been implemented, allowing for the following actions:

- **Round Robin:** Thanks to the set `TIME_QUANTUM` of 5 seconds, ensuring that no single job consumes most of the system resources.
- **Priority Scheduling:** Allows for the prioritization of kernel processes, where tasks are executed depending on their level of urgency.

File System Operations & Security

To avoid errors, system file management performs atomic operations, using temporary files that replace the originals only after the operation is successfully completed (`mv`), preventing data loss in the event of unexpected failures. In terms of security, access control with account lockout was implemented a critical concept in systems administration to prevent brute-force attacks.

Automation Techniques

In various parts of the different scripts, automation techniques are used, combining different command outputs to improve information output.

Challenges Encountered and Resolutions

- **Race Conditions:** To prevent the `job_queue.txt` file from collapsing due to multiple instances being entered simultaneously, a File Locking mechanism

was implemented. it creates a `/tmp/scheduler.lock` file at the start and releases it at the end, blocking parallel executions.

- **Data Integrity:** Users could enter delimiter characters (|) that broke the column structure. In this case, an input validation layer with regular expressions was developed to sanitize the data before storage.
- **File Identification:** Comparison by name did not detect actual duplicates if files were renamed. This task has been delegated to Python to perform a deep binary scan of each file.

4. Execution Steps and Expected Output

Task 1
Main Menu.

```
==============================================
  UNIVERSITY DATA CENTRE MONITORING SYSTEM
==============================================
1) Show current CPU and memory usage
2) List top 10 processes by memory usage
3) Terminate a process (with protection)
4) Inspect disk and archive logs
5) Bye (exit)
==============================================
Select option [1-5]:
```

Option 1

```
Select option [1-5]: 1
=== SYSTEM RESOURCE OVERVIEW ===
----------------------------------------
MEMORY SUMMARY:
Total  Used   Free   Shared  Buff/Cache  Available
8,2Gi  1,5Gi  3,6Gi  3,2Gi   6,6Gi       6,6Gi

LOAD AVERAGE (1m/5m/15m): 0,35, 0,20, 0,13
----------------------------------------
TOP 5 CPU PROCESSES (SNAPSHOT):
USER     PID    %CPU   %MEM   COMMAND
----------------------------------------
root     5103   600    0.0    ps aux --sort=-%cpu
juanflo+ 2073   14.8   5.2    /usr/bin/gnome-shell
juanflo+ 5085   5.8    0.3    /usr/libexec/tracker-extract-3 --so...
juanflo+ 2990   4.2    5.1    /snap/firefox/7967/usr/lib/firefox/...
juanflo+ 3649   1.2    3.3    /snap/firefox/7967/usr/lib/firefox/...
----------------------------------------

Press Enter to return to menu...
```

Option 2

```
Select option [1-5]: 2
=== TOP 10 PROCESSES BY MEMORY USAGE ===
USER     PID    %CPU   %MEM   COMMAND
----------------------------------------
juanflo+ 2073   14.9   5.3    /usr/bin/gnome-shell
juanflo+ 2990   4.2    5.2    /snap/firefox/7967/usr/lib/firefox/firef...
juanflo+ 3649   1.2    3.3    /snap/firefox/7967/usr/lib/firefox/firef...
juanflo+ 3234   0.8    1.7    /snap/firefox/7967/usr/lib/firefox/firef...
juanflo+ 3460   0.0    1.2    /snap/firefox/7967/usr/lib/firefox/firef...
juanflo+ 3872   0.0    0.8    /snap/firefox/7967/usr/lib/firefox/firef...
juanflo+ 3653   0.0    0.8    /snap/firefox/7967/usr/lib/firefox/firef...
juanflo+ 4388   0.0    0.8    /snap/firefox/7967/usr/lib/firefox/firef...
juanflo+ 2266   0.0    0.7    /usr/libexec/evolution-data-server/evolu...
juanflo+ 4994   0.5    0.7    gjs /usr/share/gnome-shell/extensions/di...
----------------------------------------
Press Enter to continue...
```

Option 3

```
Select option [1-5]: 3
=== TERMINATE PROCESS ===
Enter PID to terminate: 1
ERROR: Critical system process cannot be terminated.
Press Enter to continue...S
```

Option 4

```
Select option [1-5]: 4
Enter directory to inspect: /tmp
=== DISK USAGE FOR /tmp ===
77M     /tmp
Scanning for .log files larger than 50MB in /tmp...
Found large log files:
  /tmp/heavyfile.log
tar: Eliminando la `/' inicial de los nombres
Large logs archived successfully: ArchiveLogs/logs_20260314_205519.tar.gz
Press Enter to continue...
```

Task2

Main Menu

```
==============================================
  HPC JOB SCHEDULER
==============================================
1) View pending jobs
2) Submit new job
3) Process queue (Round Robin)
4) Process queue (Priority Scheduling)
5) View completed jobs
6) Exit
==============================================
Option [1-6]:
```

Option 1

```
==============================================
Option [1-6]: 1
=== PENDING JOBS QUEUE ===
ID    STUDENT_ID    JOB_NAME        EST_TIME    PRIO
1     4             luciastop       115s        1
2     10            Samsara         230s        4
3     4             testing2        345s        9
Press Enter...
```

Option 2

```
==============================================
Option [1-6]: 2
=== SUBMIT NEW JOB ===
Student ID: 4
Job name: testing2
Est. Time (s): 345
Priority [1-10]: 9
✓ Job 'testing2' submitted.
Press Enter...
```

Option 3

```
==============================================
Option [1-6]: 3
=== ROUND ROBIN PROCESSING (Q=5s) ===
Empty queue.
Press Enter...
```

Option 4

```
==============================================
Option [1-6]: 4
=== PRIORITY SCHEDULING ===
  ✓ EXECUTED (Prio 9): testing2
  ✓ EXECUTED (Prio 4): Samsara
  ✓ EXECUTED (Prio 1): luciastop
All priority jobs processed.
Press Enter...
```

Option 5

```
==============================================
Option [1-6]: 5
=== COMPLETED HISTORY ===
Student: 1          | Job: testing         | Status: OK
Student: 4          | Job: testing2        | Status: OK
Student: 10         | Job: Samsara         | Status: OK
Student: 4          | Job: luciastop       | Status: OK
Press Enter...
```

Task3

Option 1

```
1) Entregar  2) Duplicados  3) Listar  4) Login  5) Salir
Opción: 1
=== NUEVA ENTREGA ===
ID Estudiante: 1
Ruta del archivo: /home/juanflores/Documentos/AssessmentAOS/imagenprueba.png
Error: Extensión .png no permitida (Solo PDF/DOCX).

1) Entregar  2) Duplicados  3) Listar  4) Login  5) Salir
Opción:
```

Option 2

```
1) Entregar  2) Duplicados  3) Listar  4) Login  5) Salir
Opción: 3
total 496K
-r-------- 1 root root 246K mar 14 21:39 '1_20260314_213919_Assessment1 AOS-26 MSMK.pdf'
-r-------- 1 root root 246K mar 14 21:40  1_20260314_214042_copia.pdf

1) Entregar  2) Duplicados  3) Listar  4) Login  5) Salir
Opción:
```

Option 3

```
1) Entregar  2) Duplicados  3) Listar  4) Login  5) Salir
Opción: 4
=== LOGIN AL SISTEMA ===
ID Estudiante: 1
¿Resultado del login? (s/f): s
Bienvenido, 1

1) Entregar  2) Duplicados  3) Listar  4) Login  5) Salir
Opción: 4
=== LOGIN AL SISTEMA ===
ID Estudiante: 4
¿Resultado del login? (s/f): f
Fallo de login registrado.

1) Entregar  2) Duplicados  3) Listar  4) Login  5) Salir
Opción:
```

5. References (Harvard Style)

- Silberschatz, A., Galvin, P.B. and Gagne, G. (2018). *Operating System Concepts*. 10th edn. Hoboken: Wiley.
- Shotts, W. (2019). *The Linux Command Line: A Complete Introduction*. 2nd edn. San Francisco: No Starch Press.
- Tanenbaum, A.S. and Bos, H. (2014). *Modern Operating Systems*. 4th edn. Harlow: Pearson Education.